

Skill 5:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 200

void parse(char *input) {
    char token[MAX];
    int i = 0, j = 0;
    char quote = 0;

    while (input[i] != '\0') {
        if (input[i] == '"' || input[i] == '\') {
            quote = input[i++];
            j = 0;

            while (input[i] != '\0' && input[i] != quote)
                token[j++] = input[i++];

            if (input[i] == quote)
                i++;

            token[j] = '\0';
            printf("Quoted: %s\n", token);
        }

        else if (input[i] != ' ' && input[i] != '\n') {
            j = 0;

            while (input[i] != '\0' &&
                    input[i] != ' ' &&
                    input[i] != '\n' &&
                    input[i] != '\t')
                token[j++] = input[i++];

            token[j] = '\0';
            printf("Token: %s\n", token);
        }

        else {
            i++;
        }
    }
}

int main() {
    char input[MAX];

    printf("Enter input: ");
    fgets(input, MAX, stdin);

    if (strlen(input) <= 1) {
        printf("Empty input\n");
        return 0;
    }

    printf("\nParsing Result:\n");
    parse(input);

    return 0;
}
```

```
userhasini@DESKTOP-QDSCC4:~$ nano skill5.c
userhasini@DESKTOP-QDSCC4:~$ gcc skill5.c -o skill5
userhasini@DESKTOP-QDSCC4:~$ ./skill5
Enter input: hello

Parsing Result:
Token: hello
userhasini@DESKTOP-QDSCC4:~$ |
```

Skill 6:

```

GNU nano 2.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#define MAX 200

int parse(char *input, char *args[]) {
    static int argc = 0;
    static char token[MAX];
    int i = 0, k = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        if (input[i] == ' ' && input[i + 1] != '\0') {
            if (input[i] == '\\' && input[i + 1] != '\0') {
                token[k++] = input[i + 1];
                i += 2;
            }
            else if (input[i] == '`') {
                if (k > 0) {
                    token[k] = '\0';
                    args[argc++] = strdup(token);
                    k = 0;
                }
                i++;
            }
            else {
                token[k++] = input[i++];
            }
        }
        if (k > 0) {
            token[k] = '\0';
            args[argc++] = strdup(token);
        }

        args[argc] = NULL;
        return argc;
    }
}

int main() {
    char input[MAX];
    char *args[MAX];
    int count;

    printf("Enter command: ");
    fgets(input, MAX, stdin);

    count = parse(input, args);

    printf("\nParsed Arguments:\n");
    for (int i = 0; i < count; i++)
        printf("%d %s\n", i, args[i]);

    if (count == 0) {
        printf("Empty input\n");
        return 0;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process: PID = %d\n", getpid());
        execvp(args[0], args);
        perror("Execution failed");
        exit(1);
    }
    else {
        printf("Parent Process: PID = %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    for (int i = 0; i < count; i++)
        free(args[i]);

    return 0;
}

```

```

userhasini@DESKTOP-QDSCC4:~$ nano skill6.c
userhasini@DESKTOP-QDSCC4:~$ gcc skill6.c -o skill6
userhasini@DESKTOP-QDSCC4:~$ ./skill6
Enter command: ls

Parsed Arguments:
Empty input

```

Skill 7:

```

GNU nano 3.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX 100

int main() {
    char *path;
    char *dir;
    char *copy;
    char *fullpath;
    char command[MAX];

    pid_t pid;
    int status;

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH not found\n");
        return 1;
    }

    printf("Enter command to locate: ");
    scanf("%99s", command);

    copy = strdup(path);

    if (copy == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    dir = strtok(copy, ":");

    while (dir != NULL) {
        fullpath = malloc(strlen(dir) + strlen(command) + 2);

        if (fullpath == NULL) {
            free(copy);
            return 1;
        }

        sprintf(fullpath, "%s/%s", dir, command);

        if (access(fullpath, X_OK) == 0) {
            printf("Executable found: %s\n", fullpath);
            break;
        }

        free(fullpath);
        fullpath = NULL;
        dir = strtok(NULL, ":");
    }

    if (dir == NULL) {
        printf("Command not found: %s\n", command);
        free(copy);
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork");
        free(fullpath);
        free(copy);
        return 1;
    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        exec(fullpath, command, NULL);
        perror("Execution failed");
        exit(1);
    }

    printf("Parent PID: %d\n", getpid());
    printf("Waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("Child exited with status %d\n", WEXITSTATUS(status));

    free(fullpath);
    free(copy);

    return 0;
}

```

```

userhasini@DESKTOP-Q0SCCQ:~$ nano skill7.c
userhasini@DESKTOP-Q0SCCQ:~$ gcc skill7.c -o skill7
userhasini@DESKTOP-Q0SCCQ:~$ ./skill7
Enter command to locate: ls
Executable found: /usr/bin/ls
Parent PID: 698
Waiting for child...
Child PID: 701
Signal_Process_Control  childfork.c  fifo_create  file.c  hospital_shared.dat  parent  practical1.c.save  readusername  sigterm.c  skill3  skill6.c  wait  zombie.c
a.out  display  fifo_create.c  file1.txt  mutichild  parent.c  priority  readusername.c.save  simplecommandlopp.c  skill3.c  skill7  wait.c
child  display.c  fifo_read  hello  mutichild.c  pid  priority.c  roundrobin  simplecommandlopp.c  skill3.c  skill7.c  waitchild
child.c  exchp.c  fifo_read.c  hello.c  myfifo  pid.c  processid  roundrobin.c  sjf  skill4.c  socketclient  waitchild.c
childexecuting  exchp  fifo_write  hospital  orphan  pipes  processid.c  signal  sjfnonpremitive  skill5  socketclient.c  waitpid
childexecuting.c  fcfs  fifo_write.c  hospital.c  orphan.c  pipes.c  rdonly  signal.c  sjfnonpremitive  skill5.c  socketserver  waitpid.c
childfork  fcfs.c  file  hospital_scheduler.log  oslab  practical1.c  rdonly.c  sigterm  sjfnonpremitive  skill6  socketserver.c  zombie
Child exited with status 0
userhasini@DESKTOP-Q0SCCQ:~$

```